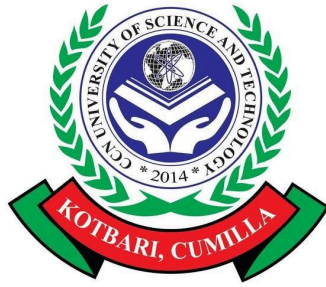


CCN University of Science & Technology



Lab Report on

Course Title: Operating System (Sessional)

Course Code:0612-1308

Experiment name: Searching and Sorting Algorithm
Implementation.

Lab no- 01

Submitted By:

Mafruda Jaman

ID-111223066

Batch-12th

Department of CSE

CCN-UST

Submitted To:

Farhana Abedin

Lecturer,

Department of

CSE

CCN-UST

Date of Submission : 24 August 2025.

Linear search : Checks each element one by one.

Works on any array (sorted or unsorted).

Time Complexity: $O(n)$

Space Complexity: $O(1)$

```
def linear_search(arr, target):
```

```
    for i in range(len(arr)):
```

```
        if arr[i] == target:
```

```
            return i
```

```
    return -1
```

Binary search : Only works on sorted arrays.

Divides search range in half each time.

Time Complexity: $O(\log n)$

Space Complexity: $O(1)$

```
def binary_search(arr, target):
```

```
    left, right = 0, len(arr)-1
```

```
    while left <= right:
```

```
        mid = (left + right) // 2
```

```
        if arr[mid] == target:
```

```
            return mid
```

```
        elif arr[mid] < target:
```

```
            left = mid + 1
```

```
        else:
```

```
            right = mid - 1
```

```
    return -1
```

Bubble sort : Repeatedly swaps adjacent elements if in wrong order.

Time Complexity: $O(n^2)$

Space Complexity: $O(1)$

```
def bubble_sort(arr):
    n = len(arr)
    for i in range(n):
        for j in range(n-i-1):
            if arr[j] > arr[j+1]:
                arr[j], arr[j+1] = arr[j+1], arr[j]
```

Merge sort : Divides array into halves, sorts them, then merges.

Time Complexity: $O(n \log n)$

Space Complexity: $O(n)$

```
def merge_sort(arr):
    if len(arr) <= 1:
        return arr
    mid = len(arr)//2
    left = merge_sort(arr[:mid]) # Recursively sort left half
    right = merge_sort(arr[mid:]) # Recursively sort right half
    return merge(left, right)
```

```
def merge(left, right):
    result = []
    i = j = 0
    while i < len(left) and j < len(right):
        if left[i] < right[j]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1
    result.extend(left[i:]) # add all remaining element
    result.extend(right[j:]) # add all remaining element
    return result
```

```
mj = [23, 5, 42, 12, 8, 19, 3]
target = 8
```

Linear Search

```
linear_result = linear_search(mj, target)
print(f"Linear Search: Target found at index {linear_result}")
```

Binary Search (requires sorted array)

```
sorted_array = merge_sort(mj.copy())
binary_result = binary_search(sorted_array, target)
print(f"Binary Search: Target found at index {binary_result} (in  
sorted array: {sorted_array})")
```

Bubble Sort

```
bubble_array = mj.copy()
bubble_sort(bubble_array)
print(f"\nBubble Sort Result: {bubble_array}")
```

Merge Sort

```
merge_array = merge_sort(mj.copy())
print(f"Merge Sort Result: {merge_array}")
```